

**Расчетно-графическая работа
для студентов заочной формы обучения**

Передача зашифрованных файлов по сети

Чтобы передать файл по сети от одного пользователя другому или организовать обмен сообщениями (чат) в зашифрованном виде, необходимо построить секретный ключ (методы DH, MQV), привести его к нужному размеру с помощью хеш-функции, зашифровать (на одной стороне), расшифровать (на другой стороне). Для шифрования рекомендуется применить потоковый шифр – это современный подход (даже блочные шифры в этом сценарии чаще всего используются в потоковых режимах).

Потоковые шифры (stream ciphers) основаны на генераторах псевдослучайных последовательностей. Псевдослучайные последовательности, применяемые в шифрах, часто называются *ключевыми потоками* (keystreams). Ключевой поток должен быть неотличим от случайного процесса, порождающего равновероятные и независимые символы (биты). При этом ключевой поток является функцией секретного ключа K и инициализирующего вектора IV . Предполагается, что ключ не меняется в течение длительного времени (сеанса), а IV должен быть разным для разных сообщений (не допускается, чтобы при шифровании различных данных использовались одинаковые ключевые потоки). IV не обязан быть секретным. Шифрование данных производится путём сложения по модулю 2 бит данных с битами ключевого потока. Расшифрование производится фактически путём повторного шифрования с тем же ключевым потоком.

В нашем распоряжении имеются авторские программы потоковых шифров – финалистов конкурса eSTREAM: HC-128, Rabbit, Salsa20 и Sosemanuk. Все они имеют одинаковый программный интерфейс. Основные элементы интерфейса следующие:

ECRYPT_ctx

Структура, описывающая внутренне состояние генератора. В программе необходимо создать экземпляр этой структуры.

void ECRYPT_init (void)

Функция инициализации. Должна вызываться в самом начале.

void ECRYPT_keysetup (ECRYPT_ctx* ctx, const u8* key,
u32 keysize, u32 ivsize)

Установка в структуру ctx секретного ключа key. Типы u8 и u32 – это соответственно байт и 32-битное слово, рассматриваемые как числа без знака. Параметры keysize и ivsize – размеры ключа и инициализирующего вектора в битах.

void ECRYPT_ivsetup (ECRYPT_ctx* ctx, const u8* iv)

Установка в структуру ctx инициализирующего вектора iv.

Эти три функции должны вызываться строго последовательно. Функция ECRYPT_ivsetup может вызываться многократно, если секретный ключ не меняется, а нужно только изменить IV .

```
void ECRYPT_encrypt_bytes (ECRYPT_ctx* ctx, const u8* plaintext,  
                           u8* ciphertext, u32 msglen)
```

Функция зашифрования (расшифрования при повторном вызове). `ctx` – текущее состояние генератора; `plaintext` – исходный текст (зашифрованный текст при расшифровании); `ciphertext` – зашифрованный текст (восстановленный текст при расшифровании); `msglen` – длина текста *в байтах*. Функция может вызываться многократно, каждый раз зашифрование (расшифрование) будет проводиться на основе продолжающейся псевдослучайной последовательности.

Для каждой программы в конце соответствующего файла *.c дописана функция `main` в качестве примера использования. Её нужно удалить, если потоковый шифр интегрируется в другое приложение. Что касается примера, то он реализует программу, которая при запуске из командной строки принимает два аргумента: ключ и инициализирующий вектор в виде слов из не более чем 16 символов (проверки на переполнение нет, если аргументы не заданы, то они полагаются равными нулю). Производится шифрование массива, заполненного нулями, и результат выдаётся на `stdout`. Фактически получается псевдослучайная последовательность, которая генерируется алгоритмом.

Материал для программной реализации **криптографических хеш-функций** содержится в каталоге HASH. Обзор хеш-функций приведен в файле RF2018-hash.pdf (это фрагмент из новой книжки Рябко Б. Я., Фионова А. Н.; из-за особенностей бесплатной программы извлечения в нем не пропечатались тире). В каталоге есть официальные документы и реализации для отечественной хеш-функции ГОСТ Р 34.11-2012 и американской хеш-функции SHA-3 (FIPS 202, Кессак). Однако для большинства студентов разобраться в официальных реализациях будет сложно. Поэтому предложены также упрощенные варианты программ, описанные ниже.

ГОСТ Р 34.11-2012 в простейшем виде (без оптимизаций) реализован в каталоге hGOST. Там, в частности, есть пример программы `test.cpp`, показывающий, как интегрировать хеш-функцию в свое приложение. Пример не нуждается в особых комментариях. Все файлы необходимо скопировать в рабочий каталог. Файл `gost3411-2012-core.c` – добавить в проект (в командную строку компилятора).

Для SHA-3 (равно как и других известных хеш-функций) можно воспользоваться простой библиотекой, содержащейся в каталоге `hash-library`. Она в обязательном порядке требует C++. Например, для интегрирования в свое приложение SHA-3 достаточно взять два файла – `sha3.h` и `sha3.cpp`. Техника использования кода описана в комментариях внутри файла `sha3.h`.

Задание

1. Для всех четырёх шифров сравнить длительность функций инициализации, и время генерации ключевого потока.
2. Выбрать одну понравившуюся хеш-функцию и интегрировать ее в программу, как это требуют протоколы выработки секретного ключа. Сравнить время вычисления хеш-функции с временем выполнения других операций в протоколе.
3. Выбрать один понравившийся потоковый шифр и реализовать на его основе приложение для передачи файла в зашифрованном виде по сети с ключом шифрования, полученным по протоколу MQV (или DH, если MQV не был сделан).
4. Провести сравнительный анализ времени выполнения отдельных шагов (фаз) алгоритма (установление ключа, шифрование, передача по сети).

Отчет о РГР должен включать в себя описание разработанной программы, текст программы, результаты экспериментов. Отчет в виде файла представляется через ЭИОС.